QALight

Frontend-розробка

Заняття 24. Типи даних

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

Порядок денний



Оголошення змінних. Структурна та номінальна типізація, перевірка типів, еквівалентність типів, статичні та динамічні типи, сильні та слабкі типи.
Приведення типів.
Практика приведення типів.

Типи даних

Оголошення змінних. Структурна та номінальна типізація, перевірка типів, еквівалентність типів, статичні та динамічні типи, сильні та слабкі типи. Приведення типів.

Примітивні типи JavaScript

Теорія: TypeScript

У JavaScript є три дуже поширені примітивні типи: string, number та boolean. Кожен із них має відповідний тип у TypeScript. Ці назви збігаються з тими, що ви побачили б, застосувавши оператор `**typeof**` мови JavaScript до значень відповідних типів:

string - для рядкових значень, наприклад "Hello, world"

number - для чисел, наприклад 42. У JavaScript немає окремого типу для цілих чисел під час виконання програми, тому тут немає аналогів int чи float - усе це просто number

boolean - для двох значень: true та false

Назви типів String, Number та Boolean (з великої літери) є допустимими, але вони позначають **спеціальні вбудовані типи**, які вкрай рідко зустрічатимуться у коді. Для визначення типів завжди використовуйте **string**, **number** або **boolean**.

Є також 2 менш поширені примітиви, які використовуються в Javascript і TypeScript.

bigint - цілі числа та значення з плаваючою комою, але допускаються більші від'ємні та додатні числа, ніж тип number.

symbol використовуються для створення глобального унікального ідентифікатора.

Призначення типу

Теорія: TypeScript

Під час створення змінної TypeScript призначає тип двома основними способами:

- Явно
- Неявно

Типи можуть використовуватися не лише в анотаціях типів. Вивчаючи самі типи, ми також дізнаємося, де саме можна на них посилатися для створення нових конструкцій.

Явний:

```
let firstName: string = "John";
```

Явні призначення типу легше читати.

Неявні – TypeScript "вгадає" тип на основі призначеного значення, це називається **infer(робити висновок)**:

```
let firstName = "John";
```

Неявне призначення змушує TypeScript виводити значення. Неявні призначення типів коротші, швидші для введення та часто використовуються під час розробки та тестування.

Анотації типів для змінних

Теорія: TypeScript

Оголошуючи змінну за допомогою `**const**`, `**var**` або `**let**`, ви можете за бажанням додати **анотацію** типу, щоб явно вказати тип цієї змінної:

```
let myName: string = "Alice";
// No type annotation needed -- 'myName' inferred as type 'string'
```

У TypeScript не використовуються оголошення на кшталт `int x = 0`, де тип вказується ліворуч; **анотації типів завжди розміщуються після елемента,** тип якого визначається.

Однак у більшості випадків це не потрібно. TypeScript намагається автоматично визначити типи у коді, де це можливо. Наприклад, тип змінної визначається на основі типу її початкового значення:

```
// Анотація типу не потрібна — для 'myName' автоматично визначається тип 'string'
let myName = "Alice";
```

Здебільшого вам не потрібно спеціально вивчати правила виведення типів.

Якщо типи даних не збігаються

Практика: TypeScript

```
// TypeScript видасть помилку, якщо типи даних не збігаються.

let first_Name: string = "John"; // тип string

// намагається повторно призначити значення іншому типу
first_Name = 33;
// Type 'number' is not assignable to type 'string'.ts(2322)  let first_Name: string

// JavaScript не видаватиме помилку для невідповідних типів.
```

Тип any

Теорія: TypeScript

У TypeScript також є спеціальний тип `**any**`, який можна використовувати, коли ви не хочете, щоб певне значення спричиняло помилки під час перевірки типів.

TypeScript не завжди може правильно визначити тип змінної. У таких випадках буде встановлено тип any, що вимкне перевірку типу.

Неявне будь-яке, оскільки JSON.parse не знає, який тип даних він повертає, тому це може бути "any"...

```
const json = JSON.parse("55");

// Більшість очікує, що json буде об’єктом, але це може бути рядок або число, як у цьому прикладі

console.log(typeof json);

// Цю поведінку можна вимкнути, увімкнувши noImplicitAny як параметр у проєкті TypeScript

tsconfig.json. Це файл конфігурації JSON для налаштування поведінки деяких TypeScript.
```

Використання any

Теорія: TypeScript

Якщо значення має тип `any`, ви можете звертатися до будь-яких його властивостей (які, своєю чергою, також матимуть тип `any`), викликати його як функцію, присвоювати його змінним інших типів (або навпаки) чи робити майже будь-що інше, що є синтаксично правильним:
let obj: any = { x: 0 };
Використання `any` вимикає подальшу перевірку типів; при цьому вважається, що ви знаєте середовище краще, ніж TypeScript.

```
let obj: any = { x: 0 };
obj.foo();
obj();
obj.bar = 100;
obj = "hello";
const n: number = obj;
```

Тип `any` стає в пригоді, коли ви не хочете записувати довгий опис типу лише для того, щоб переконати TypeScript у коректності певного рядка коду.
Якщо ви не вказуєте тип, а TypeScript не може визначити його з контексту, компілятор зазвичай вважає його типом `any`. Однак цього варто уникати, оскільки тип `any` не підлягає перевірці типів. Використовуйте прапорець компілятора `**noImplicitAny**`, щоб позначати будь-яке неявне використання `any` як помилку.

Захист від помилок типів

Теорія: TypeScript

Якщо ви спробуєте додати в масив елемент іншого типу, TypeScript одразу видасть помилку:

```
let fruits: string[] = ["яблуко", "банан"];
// ❌ Помилка: Аргумент типу 'number' не можна передати в параметр типу 'string'
fruits.push(42);
// ❌ Помилка: Тип 'boolean' не можна призначити типу 'string'
fruits[0] = true;
```

Твердження щодо типу (Type Assertions)

Теорія: TypeScript

Іноді ви можете володіти інформацією про тип значення, яку TypeScript не може визначити самостійно.

Наприклад, якщо ви використовуєте `**document.getElementById**`, TypeScript знає лише те, що метод поверне певний `**HTMLElement**`, тоді як ви можете точно знати, що на сторінці завжди буде присутній `**HTMLDivElement**` із відповідним ідентифікатором (**ID**).

У такій ситуації можна скористатися твердженням щодо типу, щоб вказати конкретніший тип:

```
const container = document.getElementById('productsContainer') as HTMLDivElement | null;
```

Твердження щодо типу видаляються компілятором і не впливають на поведінку коду під час виконання.

Також можна використовувати синтаксис із кутовими дужками:

```
const container = <HTMLDivElement>document.getElementById("productsContainer");
```

Неможливі перетворення

Теорія: TypeScript

Оскільки твердження щодо типу (type assertions) видаляються під час компіляції, перевірка типу під час виконання програми не здійснюється. У разі помилкового твердження не виникне винятку (exception) і не буде отримано значення `null`.

TypeScript дозволяє лише такі твердження щодо типу, які перетворюють тип на більш конкретний або менш конкретний варіант. Це правило запобігає «неможливим» перетворенням:

```
const x = "hello" as number;
```

Перетворення типу 'string' на тип 'number' може бути помилкою, оскільки ці типи недостатньо перетинаються. Якщо це було зроблено навмисно, спочатку перетворіть вираз на 'unknown'.

Літеральні типи

Теорія: TypeScript

Окрім загальних типів `string` і `number`, ми можемо використовувати конкретні рядки та числа як типи.
Ключові слова `var` і `let` дозволяють змінювати значення змінної, тоді як `const` - ні. Це відображається на тому, як TypeScript створює типи для літералів.

```
let changingString = "Hello World";
changingString = "Olá Mundo";
// Because `changingString` can represent any possible string, that
// is how TypeScript describes it in the type system

;let changingString: string
const constantString = "Hello World";
// Because `constantString` can only represent 1 possible string, it
// has a literal type representation

const constantString: "Hello World"
```

Numeric literal types

Теорія: TypeScript

Типи числових літералів працюють так само:

```
function compare(a: string, b: string): -1 | 0 | 1 {
 return a === b ? 0 : a > b ? 1 : -1;
}
```

`null` та `undefined`

Теорія: TypeScript

У JavaScript є два примітивні значення, що вказують на відсутність або неініціалізованість значення: `null` та `undefined`.

У TypeScript існують відповідні типи з такими ж назвами. Поведінка цих типів залежить від того, чи увімкнено опцію `**strictNullChecks**`.

Якщо опцію `strictNullChecks` вимкнено, до значень, які можуть бути `null` або `undefined`, можна звертатися як зазвичай, а самі значення `null` та `undefined` можна присвоювати властивостям будь-якого типу. Це нагадує поведінку мов, у яких відсутня перевірка на `null` (наприклад, C# або Java). Відсутність перевірки таких значень часто стає причиною багатьох помилок; ми завжди рекомендуємо вмикати `strictNullChecks`, якщо це доцільно для вашого проєкту. strictNullChecks

Коли параметр strictNullChecks увімкнено, перед використанням методів або властивостей значення, яке може бути null або undefined, необхідно перевірити його на наявність цих значень.

Звуження типу (narrowing)

Теорія: TypeScript

Так само, як ми перевіряємо наявність undefined перед використанням необов’язкової властивості, ми можемо застосувати звуження типу (narrowing), щоб перевірити значення, які можуть бути null:

```
function doSomething(x: string | null) {

 if (x === null) {
   // do nothing
 } else {
   console.log("Hello, " + x.toUpperCase());
 }
}
```

Оператор твердження, що не є нульовим

Теорія: TypeScript

У TypeScript також є спеціальний синтаксис для видалення `null` та `undefined` із типу без виконання явних перевірок. Використання символу `!` після будь-якого виразу фактично є твердженням про те, що значення не дорівнює `null` або `undefined`:

```
function liveDangerously(x?: number | null) {
 // No error
 console.log(x!.toFixed());
}
```

Як і інші оператори твердження типу (type assertions), цей оператор не змінює поведінку коду під час виконання, тому важливо використовувати `!` лише тоді, коли ви впевнені, що значення не може бути `null` або `undefined`.

Тип bigint

Теорія: TypeScript

В TypeScript bigint - це примітивний тип даних, який дозволяє працювати з цілими числами довільної довжини, що виходять за межі стандартного безпечного діапазону для типу number (більше ніж $2^{53} - 1$).

Створити значення bigint можна двома способами:

1. Додаванням суфікса n до кінця цілого числа.

2. Викликом функції BigInt().

```
// Creating a BigInt via the literal syntax
const value1: bigint = 9007199254740991n;
// Creating a bigint via the BigInt function
const value2: bigint = BigInt(9007199254740991);
const value3: bigint = BigInt("9007199254740991");
```

Типи bigint та number є абсолютно різними

Теорія: TypeScript

Типи bigint та number є абсолютно різними. Їх не можна змішувати в математичних операціях без явного приведення типів.typescript

```
let num: number = 10;
let big: bigint = 20n;

// ❌ Помилка: Operator '+' cannot be applied to types 'number' and 'bigint'.
let result = num + big;

//  Правильно: привезти до одного типу
let result1 = BigInt(num) + big; // Тип: bigint (30n)
let result2 = num + Number(big); // Тип: number (30)
```

Ділення без залишку

Теорія: TypeScript

При діленні bigint результат завжди округлюється в меншу сторону до найближчого цілого (відкидається дробова частина).

```
const result: bigint = 5n / 2n; // Результат: 2n (а не 2.5)
```

Методи вбудованого об'єкта Math (наприклад, Math.max, Math.round) не працюють із bigint.

Порівняння

Теорія: TypeScript

Ви можете порівнювати number та bigint за допомогою операторів >, <, >=, <=.

Оператор строгого порівняння === поверне false, оскільки типи відрізняються.

```
console.log(2n > 1);    // true
console.log(0n === 0);  // false
console.log(0n == 0);   // true
```

Налаштування компілятора (tsconfig.json)

Тип bigint з'явився в ECMAScript 2020. Щоб TypeScript не видавав помилок при компіляції, у вашому файлі tsconfig.json властивість target або lib має бути встановлена на версію ES2020 або новішу:

```
{
  "compilerOptions": {
    "target": "ES2020"
  }
}
```

Починаючи зі стандарту ES2020, у JavaScript існує примітивний тип для роботи з дуже великими цілими числами - `BigInt`:

Тип symbol

Теорія: TypeScript

Тип symbol у TypeScript - це примітивний тип даних, який створює унікальні та незмінні ідентифікатори.

Основні характеристики

- Унікальність: Кожне значення, створене викликом Symbol(), є унікальним, навіть якщо воно має однакову текстову мітку (опис).
- Незмінність (Immutable): Значення типу symbol не можна змінити після створення.
- Ключі об'єктів: Найчастіше символи використовуються як приховані або безпечні ключі для властивостей об'єктів (вони ігноруються циклами for...in та методами на кшталт Object.keys).

Приклад використання

```
const id1: symbol = Symbol('id');
const id2: symbol = Symbol('id');
console.log(id1 === id2); // false (бо кожен Symbol унікальний)

const user = {
 name: 'Олександр',
 [id1]: 12345
};
console.log(user[id1]); // 12345
```

Особливі різновиди symbol

Теорія: TypeScript

• unique symbol: Це підтип звичайного symbol, який можна призначати лише константам (const) або readonly static властивостям. Кожен такий символ має власний неповторний тип.

• Well-known symbols: Вбудовані символи JavaScript (наприклад, Symbol.iterator, Symbol.hasInstance), які дозволяють налаштовувати внутрішню поведінку мови для ваших об'єктів.

Сильні та слабкі типи

Теорія: TypeScript

TypeScript не ділить типи на «сильні» та «слабкі» у класичному розумінні. Натомість він поєднує статичну типізацію (типи перевіряються під час компіляції) із гнучкістю JavaScript.

Проте у спільноті TypeScript термін «слабкий тип» (Weak Type) має конкретне технічне визначення, а поняття «сили» системи типів часто розглядають через механізми суворості перевірок.

Слабкі типи (Weak Types) - Об'єкти, де всі поля опціональні (?). Захищають від передачі абсолютно сторонніх об'єктів туди, де очікується конфігурація.

Сильна типізація (Strict Mode) - Налаштування компілятора (strict: true). Мінімізує помилки в рантаймі, забороняє приховані null, undefined та неконтрольовані any.

Що таке слабкі типи (Weak Types)

Теорія: TypeScript

У TypeScript тип вважається слабким, якщо він відповідає двом умовам:

1. Він є об'єктом, у якого всі властивості є необов'язковими (позначені знаком ?).
2. Він не має сигнатури індексу (index signature) для довільних властивостей.

Приклад слабкого типу:

```
interface UserConfig {
 theme?: 'light' | 'dark';
 timeout?: number;
 retries?: number;
}
```

Оскільки всі поля необов'язкові, суто теоретично під цей тип підходить абсолютно будь-який об'єкт (наприклад, { name: "John" }). Щоб уникнути помилок, компілятор TypeScript має вбудовану перевірку слабких типів (Weak Type Detection).

Якщо ви спробуєте присвоїти слабкому типу об'єкт, який не має жодного збігу по полях, TypeScript видасть помилку:

```
const config: UserConfig = {
 adminMode: true // Помилка! Об'єкт не має спільних властивостей з типом UserConfig
};
```

Сильна (строга) типізація

Теорія: TypeScript

TypeScript можна зробити дуже «сильним» (строгим), якщо увімкнути правильні налаштування у файлі tsconfig.json. Головний прапорець для цього - "strict": true.

Ось що робить типізацію в TS дійсно сильною:

* strictNullChecks: Забороняє неявно присвоювати null або undefined іншим типам. Якщо змінна має бути числом, вона не може бути порожньою, якщо це чітко не вказано (number | null).

* noImplicitAny: Компільована помилка виникне щоразу, коли TypeScript не зможе автоматично вивести тип і захоче застосувати найслабший тип any. Це змушує розробника явно описувати типи.

* unknown замість any: Тип unknown є безпечною альтернативою any. Він каже: «тут може бути що завгодно, але ви не можете нічого зробити з цією змінною (викликати метод, додати число), поки явно не перевірите її тип».

Якщо ви працюєте над проєктом, де важлива максимальна надійність, завжди тримайте режим "strict": true увімкненим.

Практика приведення типів

Щоб запустити код у браузері, потрібно налаштувати компілятор TypeScript (tsc) та підключити згенерований JavaScript-файл до HTML. Браузери не вміють читати .ts файли напряму, тому TypeScript слугує інструментом для розробки, який перетворює код на стандартний .js.

Ініціалізація та структура проєкту

Практика: TypeScript

Переконайтеся, що у папці проєкту є файл package.json. Якщо його немає, виконайте в терміналі:

```
npm init -y
npm install -D typescript
```

Створіть правильну структуру папок (це стандарт для сучасних проєктів):

* src/ — папка для вихідного коду (тут лежить ваш index.ts із усім кодом інтернет-магазину).
* dist/ — папка, куди компілятор автоматично збиратиме готовий index.js.
* index.html — корінь вашого сайту.

Налаштування файлу tsconfig.json

Практика: TypeScript

Створіть у корені проєкту файл з назвою tsconfig.json та скопіюйте туди наступну конфігурацію. Вона оптимізована для сучасної веб-розробки (ES6+) та вмикає найсуворіший режим перевірки типів:

```
{
 "compilerOptions": {
   /* Базові налаштування */
   "target": "ES6",       // На яку версію JS перетворювати код (ES6 підтримують усі сучасні браузери)
   "module": "ESNext",                    // Використовуємо сучасні модулі (import/export)
   "lib": ["DOM", "DOM.Iterable", "ES6"],    // Підключаємо типи для браузера (window, document, fetch)
   "allowJs": true,                      // Дозволяємо додавати звичайні .js файли, якщо треба
```

Папки входу та виходу

Практика: TypeScript

```
/* Папки входу та виходу */
"outDir": "./dist",                    // Куди збирати готові .js файли
"rootDir": "./src",                    // Де лежать ваші початкові .ts файли
```

Рівень строгості перевірок

Практика: TypeScript

```
  /* Рівень строгості перевірок */
  "strict": true,                    // Вмикає всі строгі перевірки (включаючи strictNullChecks)
  "noImplicitAny": true,             // Забороняє залишати типи як "any" без явного вказання
  "strictNullChecks": true,          // Змушує перевіряти DOM-елементи на існування (if (container)
...)
```

Додатковий захист від багів

Практика: TypeScript

```
    /* Додатковий захист від багів */
    "noUnusedLocals": true,                 // Попереджає про створені, але не використані змінні
    "noUnusedParameters": true,             // Попереджає про невикористані аргументи у функціях
    "removeComments": false                 // Зберігати коментарі після компіляції (можна змінити на true)
  },
  "include": ["src/**/*"]                   // Компілятор стежитиме лише за файлами всередині папки src
}
```

Файл index.html

Практика: TypeScrit

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>TypeScript Інтернет-Магазин</title>
  <link rel="stylesheet" href="./dist/style.css">
</head>

<body>
  <!-- розмітка: #productsContainer, #orderForm, #cartContainer тощо -->

  <!-- Підключаємо ЗГЕНЕРОВАНИЙ JavaScript файл -->
  <script src="./dist/index.js"></script>
</body>
</html>
```

Явне приведення типів (Type Assertion)

Практика: TypeScript

В TypeScript явне приведення типів (Type Assertion) використовується тоді, коли ви знаєте про тип об'єкта більше, ніж сам компілятор.

За замовчуванням методи пошуку елементів у DOM (document.**getElementById**) повертають загальний базовий тип **HTMLElement** (або null, якщо елемент не знайдено).

Оскільки тип **HTMLElement** є універсальним, він не має специфічних властивостей конкретних тегів - наприклад, у нього немає властивості .**value** (як в інпутів) або .checked (як у чекбоксів).

Для того, щоб сповістити TypeScript про конкретний підтип HTML-елемента, ми використовуємо **ключове слово as**.

Як Type Assertion працює на практиці

Практика: TypeScript

Подивіться на різницю між кодом без приведення типів та з ним:

```
// ❌ Помилка TypeScript
const searchInput = document.getElementById('searchInput');
// TypeScript вважає, що searchInput має тип: HTMLElement | null

console.log(searchInput.value);
// Помилка: Property 'value' does not exist on type 'HTMLElement'.
// Помилка: 'searchInput' is possibly 'null'.

// ✅ Правильно з Type Assertion та перевіркою на null
const searchInput = document.getElementById('searchInput') as HTMLInputElement | null;
// Тепер TypeScript знає, що це або текстовий інпут, або null
if (searchInput) {
  // Властивість .value стає доступною, підказки (автодоповнення) працюють!
  console.log(searchInput.value);
}
```

Основні типи DOM-елементів у TypeScript

Залежно від того, з яким тегом в HTML ви працюєте, у TypeScript використовуються різні класи:

* **HTMLInputElement** - для текстових полів, чекбоксів, радіокнопок (<input>). Має властивості .value, .checked, .disabled.
* HTMLSelectElement - для випадаючих списків (<select>). Має властивість .value та масив .options.
* HTMLTextAreaElement - для багаторядкових текстових полів (<textarea>).
* HTMLButtonElement - для кнопок (<button>). Має властивість .disabled.
* HTMLDivElement - для блоків ( ).
* HTMLSpanElement - для рядкових елементів ().
* HTMLFormElement -0 для форм (<form>). Має метод .reset() та .submit().

Робота з колекціями елементів (querySelectorAll)

Практика: TypeScript

Коли ви шукаєте кілька елементів за класом (наприклад, чекбокси категорій), метод querySelectorAll повертає колекцію **NodeListOf<Element>**. Її також потрібно уточнити, щоб при переборі через цикл .forEach() ви мали доступ до специфічних властивостей:

```
// Приведення типу для колекції елементів
const checkboxes = document.querySelectorAll('.category-checkbox') as
NodeListOf<HTMLInputElement>;


checkboxes.forEach((cb) => {
  // TypeScript точно знає, що 'cb' — це HTMLInputElement
  console.log(cb.checked); // Працює без помилок
});
```

Універсальні дужки

Практика: TypeScript

Існує ще один синтаксис приведення типів за допомогою кутових дужок <>, але його не рекомендується використовувати у веб-розробці, оскільки він конфліктує із синтаксисом JSX у React:

```
// Менш популярний синтаксис (краще уникати в сучасних проєктах):
const myInput = <HTMLInputElement>document.getElementById('searchInput');
console.log(myInput.value);
```

Глобальні змінні стану додатка

Практика: TypeScript

```
let products: Product[] = []; // Чітко вказуємо, що це масив об'єктів Product
let currentPage: number = 1;
const itemsPerPage: number = 3;
```

DOM Елементи з явним приведенням типів

Практика: TypeScript

```
// Використовуємо 'as HTMLInputElement' тощо, щоб мати доступ до .value чи .checked

const container = document.getElementById('productsContainer') as HTMLDivElement | null;
const searchInput = document.getElementById('searchInput') as HTMLInputElement | null;
const checkboxes = document.querySelectorAll('.category-checkbox') as
NodeListOf<HTMLInputElement>;
const priceMinInput = document.getElementById('priceMin') as HTMLInputElement | null;
const priceMaxInput = document.getElementById('priceMax') as HTMLInputElement | null;
const sortSelect = document.getElementById('sortSelect') as HTMLSelectElement | null;
const clearFiltersBtn = document.getElementById('clearFiltersBtn') as HTMLButtonElement | null;
const paginationContainer = document.getElementById('paginationContainer') as HTMLDivElement | null;
const totalProductsCountHTML = document.getElementById('totalProductsCount') as HTMLSpanElement
| null;
```

Компіляція та запуск розробки

Практика: TypeScript

Щоб не запускати компілятор вручну після кожної зміни коду, використовуйте режим спостереження (Watch mode). Відкрийте термінал і виконайте:

npx tsc --watch

Як це працює:

1. Компілятор TypeScript сканує папку src/, знаходить там index.ts.
2. Перевіряє код на помилки типів згідно з правилами tsconfig.json.
3. Якщо все добре, створює папку dist/ і записує туди чистий index.js.
4. Термінал залишається активним. Щойно ви натиснете Ctrl + S у файлі .ts, збірка миттєво оновиться.

Запуск у браузері

Практика: TypeScript

Оскільки ми використовуємо type="module", звичайне відкриття HTML-файлу подвійним кліком мишки з диску (file://...) не працюватиме через політику безпеки браузерів (CORS).
Вам потрібно запустити локальний сервер. Найпростіші варіанти:

* Якщо ви працюєте у VS Code, встановіть розширення Live Server і натисніть кнопку "Go Live" внизу екрана.
* Або запустіть сервер через Node.js у сусідньому вікні терміналу: npx serve або npx http-server.

ВПРАВИ



Вправи до цього заняття знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

Посилання на довідники



Typescriptlang: https://www.typescriptlang.org/

Tsconfig: https://www.typescriptlang.org/tsconfig/

Microsoft for JavaScript developers: https://developer.microsoft.com/en-us/javascript/

JavaScript Guide: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

Nodejs:](https://nodejs.org/uk)

Дякую за увагу!

До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua